

TECHPIO | AUTOTASK INTEGRATION

The Friendly Guide

One helpdesk, two systems, zero double work — how your Autotask ↔ osTicket sync works and how to use it.

Plugin schema version 2.1.2 · Generated July 16, 2026

Five short chapters for everyday use · five appendices for the technically curious.

Contents

1	Welcome
2	How it works — one ticket's story
3	Your day with a ticket
4	Setting it up
5	Questions & answers

FOR THE TECHNICALLY CURIOUS

A	Scheduler (Cron) Setup
B	Upgrades & Migrations
C	Security Notes
D	Autotask API Notes
E	Database Schema

Welcome

What this integration does — in one sentence

Your clients work in Autotask. Your team works in osTicket. This plugin keeps both sides perfectly in sync — so nobody ever has to ask "which system has the latest update?"

The promise

Whatever happens on one side shows up on the other, looking completely natural:

When someone...	The other side sees...
Creates a ticket in Autotask	The same ticket appears in osTicket — same subject, customer, priority, status, even the attached files
Replies from osTicket	A normal reply on the Autotask ticket — no robot signatures, no "sent by osTicket"
Logs 30 minutes of billable time	0.50 h, billable, with the right work type — on both sides, to the minute
Changes the status or priority	The same change, with the same name, on the other side
Uploads a file	The actual file, downloadable on both sides

And just as important — what the client **never** sees: your internal notes stay internal, your team structure stays private, and no message ever looks like it came from a machine.

Who should read what

You are...	Read...
A technician answering tickets	Chapter 3 — <i>Your day with a ticket</i> (10 minutes, everything you need)
The administrator	Chapter 4 — <i>Setting it up</i>
A manager wanting the big picture	Chapters 1 and 2
Curious when something looks odd	Chapter 5 — <i>Questions & answers</i>
A developer or auditor	The appendices at the back

 **The short version for technicians:** work the ticket in osTicket exactly like any other ticket. Fill in **Time Spent** when you reply. That's it — the plugin does everything else.

How it works — one ticket's story

Meet the cast we'll follow through this whole guide:

- **Magnolia Dental Services** — your client. They use **Autotask**.
- **Sarah** — Magnolia's front-desk manager. She reports problems.
- **Basit** — the technician *assigned* to Magnolia's tickets in Autotask.
- **Dalbeir** — one of *your* technicians, working in **osTicket**.

Monday, 9:02 — Sarah has a problem

Patients in the waiting area keep losing Wi-Fi. Sarah creates a ticket in Autotask, like she always does:

```
Title:      Wi-Fi keeps dropping in the waiting area
Priority:    Medium          Status: New          Assigned to: Basit
Attached:    wifi-complaints.png
```

9:03 — the ticket appears in osTicket

Within one sync cycle, the ticket lands in your osTicket helpdesk — routed straight to the department you chose for Magnolia. Everything came along:

- Subject, description, and Sarah as the customer (her real email)
- Priority **Medium** arrives as **Normal** (the plugin translates the vocabulary: Medium→Normal, Critical→Emergency)
- Status **New**, the attached screenshot as a real downloadable file
- A blue **TECHPIO | Autotask** panel on the ticket showing the Autotask number, the client, and an **Open in Autotask** button

9:40 — Dalbeir fixes it and logs his time

Dalbeir remotes in, finds the neighbouring café's Wi-Fi on the same channel, moves the access point to channel 11. Then he replies **in osTicket, on the normal reply form**:

```
Reply:      "I moved your guest network to channel 11 – the café next
              door was interfering. Please watch it today."
Time Spent: 30   (Minutes)
Time Type:   Remote Support
Billable:    ✓
```

He clicks Post Reply. Done. Nothing else to fill in, no second system to open.

9:41 — what Magnolia sees in Autotask

- Sarah gets the reply — a completely normal, human reply.
- A time entry appears: **0.50 h, Billable, Remote Support** — attributed to **Basit**, the ticket's assigned technician. Magnolia sees *their* technician handling *their* ticket. Your internal team stays your business.

- Meanwhile in osTicket, the 🕒 popup honestly records that **Dalbeir** did the work, marked with an **osTicket** source chip.

Tuesday — closing the loop

Basit adds his own note and a 15-minute follow-up entry *from the Autotask side* — both appear in the osTicket thread within a minute, under Basit's name, at the real time he wrote them. Sarah confirms everything works, and the ticket is set to **Complete** in Autotask. The osTicket copy closes itself.

Total time on the ticket: 0.75 h (0.50 billable) — identical on both sides, ready for invoicing.

The rules of the game (worth remembering)

1. **Autotask is the front desk.** Clients create and assign tickets there, using their own queues and technicians.
2. **osTicket is the workshop.** Any of your technicians can pick up any imported ticket and work it.
3. **Everything a client should see syncs out; nothing private leaks.** Internal notes, system notices and your team's names stay inside.
4. **Nothing ever duplicates.** Every reply, time entry and file is tracked by ID — you can sync a hundred times and get exactly one copy.

Your day with a ticket

Everything in this chapter happens on the normal osTicket ticket page — you never need to open Autotask unless you want to.

The blue panel

Every synced ticket has a blue **TECHPIO | Autotask** strip above the reply button. It tells you at a glance:

You see	It means
Autotask #T20260714.0085	The client-side ticket number
 0.75 h	Total time logged so far — click it for the full list
 Magnolia Dental Services	Which client this belongs to
 ProApps Techpio (email)	The customer contact
AUTOTASK STATUS: New	Live status on the client side
Open in Autotask ↗	Jump to the same ticket in Autotask

Replying

Type your reply exactly as you always have. It arrives in Autotask as a normal public note the customer can read. **Internal notes stay internal** on both sides — the customer never sees them.

Logging time (the part that matters for billing)

Fill the three little fields under your reply before you post:

Time Spent: 45

← minutes

Time Type: Remote Support

Billable: ✓

Minutes convert to decimal hours automatically — always exact:

You enter (minutes)	Autotask shows
10	0.17 h
15	0.25 h
30	0.50 h
45	0.75 h
60	1.00 h
90	1.50 h

What happens automatically:

- Autotask gets a **0.75 h billable Remote Support** entry carrying your reply text — attributed to the ticket's assigned technician, so the client sees a familiar name.
- The **Time Type list only shows types that exist for this client** — and it pre-selects the work type the ticket was created with. Usually you don't have to touch it.
- Your own name stays on the entry inside osTicket (🕒 popup, *Technician* column), so internal reports know who really did the work.

💡 Logging a quick phone call? `Time Spent: 10`, type stays as suggested, untick Billable if it's goodwill. Ten seconds, perfect books.

The 🕒 time popup

Click the hours in the blue panel to see every entry from **both** systems:

Date	Technician	Hours	Billable	Source	Status
2026-07-13 21:41	Dalbeir Singh	0.75	✓	osTicket	synced (AT #63)
2026-07-13 20:37	Basit Lone	0.50	✓	Autotask	synced (AT #43)

Orange chip = logged here, blue chip = logged in Autotask. The total at the bottom always matches Autotask's Time Summary.

Files

Drag a file onto your reply — it appears in Autotask's attachment list as a real file (up to ~6 MB). Files the client attaches in Autotask show up in the thread as downloadable attachments, under the uploader's name.

Statuses

The **Ticket Status** dropdown shows the client's own Autotask statuses — *New, In Progress, Waiting Customer, On Hold, Complete...* Pick one and both sides update. Two things to know:

- Setting **Complete** (or closing the ticket) completes it in Autotask, including your last reply as the resolution.
- If the ticket was completed **from Autotask**, that wins — the client's system is the system of record.

⚠️ **Close with time.** If you close a ticket without any time logged, the closure is flagged in the audit log — the yellow warning under the reply form reminds you.

Setting it up

A fresh install takes about fifteen minutes. Here's the whole journey.

Step 1 — copy the plugin in

Put the plugin folder under `include/plugins/` of your osTicket, and copy the four small "endpoint" files into `scp/` (they power the blue panel, the time fields and the docs link):

```
cp include/plugins/<plugin-dir>/scp-panel.php      scp/autotask-panel.php
cp include/plugins/<plugin-dir>/scp-autotask.php    scp/autotask.php
cp include/plugins/<plugin-dir>/scp-timefields.php  scp/autotask-timefields.php
cp include/plugins/<plugin-dir>/scp-docs.php        scp/autotask-docs.php
```

⚠ **Remember this for updates too.** These copies don't update themselves — re-copy them whenever you update the plugin. (If a change "doesn't show up", a stale copy here is the first thing to check.)

Step 2 — install and enable

Admin Panel → **Manage** → **Plugins** → **Add New Plugin** → **Autotask Integration** → **Install**, then enable it. The plugin creates and upgrades its own database tables automatically — no SQL work.

Step 3 — register your first client

Open the **Clients** page and click **Add Client**. You need four things from the client's Autotask admin:

Name:	Magnolia Dental Services	
Code:	MAGNOLIA	← the little badge on the blue panel
API Username:	api-user@magnolia...	
API Secret:	●●●●●●●●	← encrypted, never shown again
Integration Code:	XXXXXXXX	
Department:	Support	← where their tickets land in osTicket

Saving runs a **live connection test** — bad credentials simply won't save. On success the plugin also mirrors the client's statuses and work types into osTicket, so your dropdowns instantly speak their language.

Optional: route queues to different departments

By default every imported ticket lands in the client's one department. If your teams are split — NOC, Help Desk, Cyber security — click + **Add queue rule** under *Department routing by Autotask Queue* and map each Autotask queue to its own osTicket department:

29682833 (Level I Support)	→	Support / NOC
29682969 (Level II Support)	→	Support / Help Desk
(everything else)	→	the Fallback Department

Queue IDs are right there in the ID Reference panel. Rules apply when the ticket is imported; add or remove rules any time with + and X.

💡 **Looking for a "map technicians" step? There isn't one — on purpose.** Tickets are scoped by import filters (Step 4), any of your technicians can work them, and time is credited to the client's assigned resource. Nothing to map, nothing to maintain. See Chapter 5 for the full reasoning.

Step 4 — choose what to import

Import filters decide which Autotask tickets become osTicket tickets. Example: *"only the Service Desk queue, only open tickets, last 7 days"*:

```
Queue IDs:      29682833
Import open:    ✓      Import closed:  X
Import window:  7 days
```

The **ID Reference** panel right on the form lists the client's live queue, resource and company IDs — no hunting through Autotask.

Step 5 — set the defaults

When osTicket pushes a new ticket to Autotask, these fill the blanks: default company, queue, priority, work type, and the resource + role used when a ticket has no assignee yet. Pick them from the same ID Reference panel.

Step 6 — turn on the scheduler and verify

Set up the cron job (Appendix A has the one-liner), then run the built-in health check:

```
php include/plugins/<plugin-dir>/selftest.php
```

27 green checks and exit code 0 = you're live. Create a test ticket in osTicket, watch it appear in Autotask a minute later, and take the rest of the day off.

The options, translated

Option	Plain meaning
Two-way sync	osTicket changes flow back to Autotask. Leave on.
Auto-import	Autotask tickets flow in on schedule. Leave on.
Inbound notes	Replies/notes/time from Autotask appear in the thread. Leave on.
Sync attachments	Files travel both ways. Leave on.
Import system notes	Autotask's robot notes ("ticket forwarded..."). Leave off — it's noise.
Require time before close	Nags technicians who close without logging time. Recommended.
Close osTicket on Complete	Client completes it → your copy closes too. Recommended.
Status Map	Leave empty — statuses match by name automatically.

Questions & answers

The questions real users actually ask, with straight answers.

"The client says they replied, but I don't see it"

Give it one sync cycle (usually under a minute). Still nothing? Check the **Dashboard** — is the client's connection badge green and is *Last Sync* advancing? If the badge is red, the credentials or the network to Autotask are the problem, and the log line will say which.

"I logged 30 minutes but Autotask shows something else"

It shouldn't — 30 minutes arrives as exactly 0.50 h. If you see an old ticket where hours look shifted or doubled, it predates the current version; new entries are exact to four decimals. (And yes, "2 hours" as `120` minutes is fine.)

"Why does Autotask credit the time to Basit when Dalbeir did the work?"

By design. The client sees their **assigned technician** — you don't expose your internal team. Open the ⌚ popup on the osTicket side to see who *really* worked: the Technician column and the orange/blue source chips tell the true story for your own reporting.

"The dropdown is missing a status / has one I don't want"

The Status and Time Type dropdowns mirror the client's Autotask lists — if their admin adds or renames one, press ↻ Refresh on the client's edit page and osTicket follows. *Open* and *Closed* are the only extras; osTicket itself requires them and they still behave correctly (Closed completes the Autotask ticket).

"Do ticket assignments sync between the systems?"

No — by design. Autotask is the system of record: clients create and assign tickets there using their own queues and resources. osTicket imports by the configured filters (queues, companies, resources) — *not* by matching technicians — and any of your technicians can work any imported ticket. Their updates flow back to the Autotask ticket, credited to its assigned resource. There is no technician mapping to set up or maintain.

"A file didn't arrive"

Check, in order: **Sync attachments** is on for that client → the file is under Autotask's ~6 MB API limit (bigger files get a link-note in the thread instead, so nothing is lost) → the Dashboard log for an upload error on that ticket.

"Someone edited a time entry in Autotask — what happens here?"

The ⌚ popup and totals update to the new duration/billable flag. The original thread entry stays as history — billing records are never silently rewritten. Deleted in Autotask? Same: popup reflects it, history stays.

"Could a sync hiccup create duplicates?"

No. Every synced ticket stores both its Autotask ID and its osTicket ID, so updates always target the existing ticket — and every reply, note, time entry and file carries its own fingerprint ID. Run a sync a hundred times; you get

exactly one of each. This is tested to paranoia levels (the test suite syncs everything three times and counts).

"How do I verify the integration is working?"

The five-minute confidence check, end to end:

1. Create a ticket in Autotask → it appears in osTicket with the customer, priority, status and any files.
2. Reply from osTicket → the reply shows on the Autotask ticket.
3. Log **30 minutes billable** and **15 minutes non-billable** → Autotask shows 0.50 h billable and 0.25 h non-billable, and the 🕒 popup total matches Autotask's Time Summary.
4. Attach a file on each side → it's downloadable on the other.
5. Change the status in Autotask → the osTicket status follows by name (and completing the ticket closes it here).

For a deeper check, run `selftest.php` — 27 automated verdicts including a live API round-trip.

"The blue panel / a new feature isn't showing"

Hard-refresh first (Ctrl+F5). If it's still old, the `scp/` endpoint copies are stale — re-copy them (Setup, Step 1) and refresh again.

"Something is genuinely wrong — where do I look?"

1. **Dashboard** → **Recent Logs**, filter *Errors*. The messages are written for humans and name the ticket involved.
2. **Failed & Dead Jobs** on the same page — one click retries them after you fix the cause.
3. Still stuck? Run the health check and read its verdicts: `php include/plugins/<plugin-dir>/selftest.php`
4. For deep dives, set Log Level to *debug* temporarily — payloads are recorded with secrets automatically blanked out.

Scheduler (Cron) Setup

The plugin's two-way (inbound) sync, retry queue and housekeeping run on a schedule. There are **two supported ways** to drive it. Use one.

The inbound pull is throttled to the **Scheduled Sync Interval** you chose in config (default 5 min); the outbound retry queue drains on every tick.

Option A — osTicket's built-in cron (recommended)

osTicket fires a `cron` signal which this plugin handles automatically. You just need osTicket's own cron to run frequently.

A1. System crontab (every 5 minutes)

```
*/5 * * * * php /var/www/osticket/api/cron.php
```

A2. HTTP cron (if you can't use CLI)

```
*/5 * * * * curl -s "https://<helpdesk>/api/cron.php" >/dev/null
```

The interval here only needs to be $\leq$ your configured sync interval. Running osTicket cron every 5 minutes is the usual recommendation.

Option B — Dedicated plugin cron runner

Point system cron straight at the plugin's `cron.php`. Useful if you want the integration to run on its own cadence independent of osTicket cron.

```
# Incremental sync every 5 minutes
*/5 * * * * php /var/www/osticket/include/plugins/autotask-plugin/cron.php

# Optional: a nightly full sync (30-day lookback) at 02:15
15 2 * * * php /var/www/osticket/include/plugins/autotask-plugin/cron.php full
```

Web invocation (guarded)

If you must trigger it over HTTP, set a shared secret and pass it:

```
export AUTOTASK_CRON_SECRET="a-long-random-string" # in the PHP environment
```

```
*/5 * * * * curl -s "https://<helpdesk>/include/plugins/autotask-plugin/cron.php?key=a-long-random-string" >/dev/null
```

Requests without the correct `key` are rejected with HTTP 403.

Windows Task Scheduler

```
# Every 5 minutes, incremental sync
$action = New-ScheduledTaskAction -Execute "C:\php\php.exe" `
        -Argument "C:\inetpub\osticket\include\plugins\autotask-plugin\cron.php"
$trigger = New-ScheduledTaskTrigger -Once -At (Get-Date) `
        -RepetitionInterval (New-TimeSpan -Minutes 5)
Register-ScheduledTask -TaskName "Autotask Sync" -Action $action -Trigger $trigger
```

Manual & full sync

From the **Dashboard** you can run **Incremental** or **Full** sync on demand — no cron required for one-off runs.

Verifying cron

- Dashboard → **Last Inbound Sync** timestamp should advance each interval.
- Logs (category `scheduler`) record each incremental run with counts.
- Stuck `processing` jobs are auto-reaped after 10 minutes.

Upgrades & Migrations

The plugin upgrades cleanly in place. Schema changes are applied automatically by the **incremental migration runner**.

How upgrades work

- The shipped code declares `Installer::SCHEMA_VERSION`.
- On bootstrap the plugin compares it with the `schema_version` stored in the plugin config. If they differ, it scans `migrations/*.sql` and applies any files not yet recorded in `ost_autotask_migrations`, then records the new version.
- Already-applied migrations are skipped, so re-running is safe (idempotent).

Standard upgrade procedure

1. **Back up your database** (always).
2. Replace the plugin folder under `include/plugins/` with the new version. Keep your existing data tables — do **not** drop them.
3. **Re-copy the webroot endpoints** (they do NOT update themselves): `scp-panel.php` → `scp/autotask-panel.php`, `scp-autotask.php` → `scp/autotask.php` (see INSTALLATION.md §2b).
4. Load any SCP page (or run cron). The migration runner executes pending migrations on the next bootstrap.
5. Run `php <plugin-dir>/selftest.php` — require exit 0.
6. Open the **Dashboard**, confirm the green connection badge per client, and run one **Incremental Sync** to confirm health.

Migration highlights

Schema	Migration	What / why
2.x	0005_multi_tenant.sql	autotask_instance table; instance_id on all data tables.
2.1.x	0006_picklist_per_instance.sql	Picklist cache scoped per client.
2.1.2	0007_note_map_unique_scope.sql	Important fix: dedupe uniqueness re-scoped to (instance, record type, id). Autotask notes / time entries / attachments are separate id-spaces per tenant; the old global key silently dropped dedupe markers on collisions, causing repeated attachment re-imports.

Adding a new migration (for maintainers)

1. Create `migrations/000N_description.sql` (use `%PREFIX%` for the table prefix; one statement per `;`, comment lines start with `--`).
2. Bump `Installer::SCHEMA_VERSION` in `class.installer.php`.
3. Ship. The new file applies automatically; older installs catch up incrementally.

Write migrations to be **additive and idempotent** where possible (`CREATE TABLE IF NOT EXISTS`, `ADD COLUMN` guarded by checks) so partial failures can be re-run.

Rolling back

There is no automatic down-migration. To roll back:

- 1. Restore the previous plugin folder.
- 2. Restore the database from your pre-upgrade backup if a migration must be reverted.

Version compatibility

Plugin	osTicket	PHP
1.0.x	1.17 – 1.18	8.0 – 8.3

After a **major osTicket upgrade**, re-test connection and run a manual incremental sync, since core signal/ORM behaviour can change between releases.

Security Notes

Result of the pre-release security review (2026-07-14). Verdict: **PASS** — no critical or high findings. This page records the security posture and the operational hardening expectations.

Authentication & authorization

- Ticket-panel endpoints (`scp/autotask-panel.php` , `scp/autotask.php`) require an authenticated **staff** session (`staff.inc.php`) and verify **per-ticket access** (`Ticket::checkStaffPerm`) before returning context or accepting actions.
- The admin Dashboard / Clients pages require **administrator** privilege (`$thisstaff->isAdmin()`), enforced before any rendering or POST handling.
- `cron.php` and `selftest.php` are **CLI-only** (`PHP_SAPI` guard).

CSRF

Every state-changing action validates the osTicket CSRF token: admin POST handlers (`clients` , `dashboard`) and all panel actions (`log_time` , `push_ticket` , `set_status` , `complete`). Read-only `context` is GET/no-token by design.

Injection

- **SQL**: all dynamic values pass through `db_input()` or explicit `(int)` casts; review found no raw request variables reaching `db_query()` .
- **XSS**: admin templates escape through a single `$e()` helper (`htmlspecialchars` , `ENT_QUOTES`); panel JSON is emitted with `Content-Type: application/json` (never inline in HTML); the panel JS escapes all interpolated values through `esc()` .
- No `eval` / `exec` / dynamic includes anywhere in the plugin.
- Inbound Autotask content (notes, descriptions) is stored through osTicket's thread API and sanitized by core on render.

Secrets

- Client API secrets are **encrypted at rest** with osTicket's Crypto keyed by `SECRET_SALT` (subkeyed per instance store). A base64-marked fallback exists only for the rare case Crypto is unavailable — ensure `SECRET_SALT` is set (standard in every osTicket install) so the real cipher is used.
- Secrets are **never re-echoed** into forms (password field, empty value).
- The logger **redacts** secret-shaped values from all persisted payloads.

Transport

All Autotask API calls use cURL with `CURLOPT_SSL_VERIFYPEER = true` and `CURLOPT_SSL_VERIFYHOST = 2` . Zone URLs come from Autotask zone detection — never from request input.

Files

- Outbound uploads capped at ~6 MB (Autotask API limit), read through the osTicket file API.
- Inbound files are created through `AttachmentFile::create` (osTicket's content-addressed store);
oversize/withheld binaries degrade to a link note.

Operational expectations

- Run `selftest.php` after every deploy/upgrade (exit 0 gate).
- Keep `logs/` non-web-readable (default perms from INSTALLATION.md §2).
- The webroot endpoints in `scp/` must only ever be the shipped copies (see INSTALLATION.md §2b).
- Database credentials/`SECRET_SALT` hygiene is inherited from the osTicket installation itself.

Autotask API Notes

The plugin talks to the **Autotask PSA REST API v1.0** using native cURL. All access is encapsulated in `class.api.php` (`Autotask\AutotaskApi`).

Authentication

Three HTTP headers are sent on every authenticated request:

```
ApiIntegrationcode: <integration code>
UserName:           <api username>
Secret:             <api secret>
Content-Type:       application/json
```

Zone detection

If no Zone URL is configured, the client resolves it once via:

```
GET https://webservices.autotask.net/at-services-rest/v1.0/zoneInformation?user=<username>
```

The returned `url` is normalised to the versioned API base, e.g.

`https://webservices2.autotask.net/at-services-rest/v1.0/`, and cached back into the plugin config on successful save.

Public methods (`Autotask\AutotaskApi`)

Method	Endpoint	Purpose
testConnection()	Companies/query (MaxRecords 1)	Validate creds; returns {ok, message, zone_url}
resolveZone()	zoneInformation	Detect/normalise zone base
createTicket(array \$fields)	POST Tickets	Returns new ticket id
updateTicket(int \$id, array \$fields)	PATCH Tickets	Partial update
getTicket(int \$id)	GET Tickets/{id}	Single ticket
getTicketsModifiedSince(string \$utc, int \$max)	GET Tickets/query	Inbound delta (filter lastActivityDate gte)
createTicketNote(int \$ticketId, ...)	POST Tickets/{id}/Notes	Add note (publish 1=all, 2=internal)
getTicketNotesSince(int \$ticketId, string \$utc)	GET Tickets/{id}/Notes/query	Inbound note delta
findContactByEmail(string \$email)	GET Contacts/query	Resolve contact + company
findCompanyByName(string \$name)	GET Companies/query	Resolve company
getFieldInfo(string \$entity)	GET {entity}/entityInformation/fields	Picklist metadata (look up status/priority/queue IDs)

Query filter shape

Autotask `query` endpoints accept a URL-encoded JSON `search` parameter:

```
{
  "filter": [
    { "op": "gte", "field": "lastActivityDate", "value": "2026-06-30T00:00:00Z" }
  ],
  "MaxRecords": 50
}
```

Error handling & rate limiting

- Non-2xx responses raise `Autotask\ApiException` carrying the HTTP status and a `retryable` flag.
- `401/403` → authentication failure (non-retryable, logged under `auth`).
- `429/503` → one short inline retry honouring `Retry-After` (bounded to 5s); persistent failures defer to the retry queue with exponential backoff.
- Transport errors (cURL) → retryable.

Looking up picklist IDs (recipe)

From the dashboard host, a quick PHP one-off:

```

$api = new \Autotask\AutotaskApi($creds);
foreach ($api->getFieldInfo('Tickets') as $f) {
    if (in_array($f['name'], ['status','priority','queueID','ticketType'])) {
        echo $f['name'], ":\n";
        foreach ($f['picklistValues'] ?? [] as $pv) {
            echo "    {$pv['value']} => {$pv['label']}\n";
        }
    }
}
}

```

Attachment binaries — endpoint quirk

The **list** endpoint (`GET Tickets/{id}/Attachments`) always returns `data: ""`. The binary is only returned by the **by-id** endpoint (`GET Tickets/{id}/Attachments/{attachmentId}`). Inbound attachment sync therefore lists first (metadata + ids), then fetches each new file by id. Autotask caps attachment uploads at ~6 MB.

Service-ticket hours quirk

On Service Desk tickets Autotask derives `hoursWorked` from the `startDateTime` / `endDateTime` window; a PATCH that changes only `hoursWorked` is ignored. Edits must move the window.

Database Schema

All tables use the osTicket `TABLE_PREFIX` (shown here as `ost_`), InnoDB and `utf8mb4`. They are created automatically by the migration runner (`class.installer.php`); the full DDL snapshot lives in `sql/install.sql`.

ost_autotask_migrations

Tracks applied migration files for incremental upgrades.

Column	Type	Notes
id	INT UNSIGNED PK	
filename	VARCHAR(191) UNIQUE	e.g. 0001_initial_schema.sql
applied_at	DATETIME	

ost_autotask_settings

Runtime key/value store (sync cursors, last-run timestamps, cached test result). Distinct from encrypted credentials, which live in osTicket's PluginConfig.

Column	Type	Notes
id	INT UNSIGNED PK	
skey	VARCHAR(191) UNIQUE	e.g. inbound_cursor_utc
svalue	LONGTEXT	JSON-encoded
updated	DATETIME	

ost_autotask_ticket_map *(Ticket Mapping)*

One row per synced ticket — the heart of the integration.

Column	Type	Notes
id	INT UNSIGNED PK	
osticket_ticket_id	INT UNSIGNED UNIQUE	
autotask_ticket_id	BIGINT UNSIGNED (idx)	NULL until created
autotask_ticket_number	VARCHAR(64)	
last_sync_time	DATETIME (idx)	
last_updated_by	ENUM-ish VARCHAR(16)	
sync_direction	ENUM	to_autotask / to_osticket / bidirectional
osticket_hash	VARCHAR(64)	change-detection hash
autotask_lastactivity	DATETIME	inbound cursor per ticket
created / updated	DATETIME	

ost_autotask_sync_queue *(Sync / Retry Queue)*

Durable outbound jobs with exponential backoff.

Column	Type	Notes
id	BIGINT UNSIGNED PK	
osticket_ticket_id	INT UNSIGNED (idx)	
entity_type	VARCHAR(32)	ticket/note/reply/status/priority
action	VARCHAR(32)	create/update/append
payload	LONGTEXT	JSON
status	ENUM	pending/processing/done/failed/dead
attempts	SMALLINT UNSIGNED	
next_attempt_at	DATETIME	indexed with status
last_error	TEXT	
dedupe_key	VARCHAR(191) UNIQUE	collapses duplicate pending jobs
created / updated	DATETIME	

Index: idx_status_next (status, next_attempt_at) powers efficient claiming.

ost_autotask_log *(Error / API Logs)*

Column	Type	Notes
id	BIGINT UNSIGNED PK	
level	ENUM	debug/info/warning/error (idx)
category	VARCHAR(64) (idx)	api/auth/outbound/inbound/queue/...
osticket_ticket_id	INT UNSIGNED	
autotask_ticket_id	BIGINT UNSIGNED	
message	TEXT	
request / response	MEDIUMTEXT	captured at debug, secrets redacted
created	DATETIME (idx)	

`ost_autotask_sync_history` **(Sync History)**

Column	Type	Notes
id	BIGINT UNSIGNED PK	
map_id	INT UNSIGNED (idx)	→ ticket_map.id
osticket_ticket_id / autotask_ticket_id		
direction	ENUM	to_autotask / to_osticket
entity_type	VARCHAR(32)	
status	ENUM	success/failed/skipped (idx)
summary	VARCHAR(255)	
created	DATETIME (idx)	

Enterprise foundation tables (migration 0003)

Relationships to `ost_autotask_ticket_map` are **logical** (indexed columns, no enforced FK constraints) to match osTicket's schema style.

`ost_autotask_note_map`

Two-way dedupe map between osTicket thread entries and Autotask records. Key columns: `instance_id`, `ticket_map_id`, `osticket_entry_id`, `autotask_note_id`, `direction`, `note_type` (`note` / `reply` / `timeentry` / `attachment`).

Since schema **2.1.2** (migration `0007`) the unique keys are (`instance_id`, `note_type`, `autotask_note_id`) and (`instance_id`, `note_type`, `osticket_entry_id`) — Autotask notes, time entries and attachments are separate id-spaces, per tenant; a global key silently swallowed dedupe markers on id collisions.

`ost_autotask_time_entry`

Time entries created/synced for a mapped ticket. Maps to Autotask `TimeEntries`: `hours_worked`, `start/end_datetime`, `billing_code_id` (work type), `role_id`, `billable`, `summary_notes`, `internal_notes`,

`status`, `created_by` (staff id).

`ost_autotask_picklist_cache`

Cached Autotask picklist values for performance + pre-submit validation. Unique on (`entity`, `field`, `value`); stores `label`, `is_active`, `is_default`.

`ost_autotask_audit`

Privileged-action audit trail / change tracking: `staff_id`, `action` (e.g. `time.add`, `ticket.close`), `osticket/autotask_ticket_id`, `detail` (JSON), `ip`.

`ost_autotask_import_filter`

Saved import filter sets (admin checkboxes). `criteria` JSON holds statuses, queues, companies, resources, unassigned flag, date range, include-closed.

`ost_autotask_conflict`

Field-level conflicts awaiting resolution: `field`, `osticket_value`, `autotask_value`, `resolution` (`unresolved` / `autotask_wins` / `osticket_wins` / `manual`).

Entity relationships

```
ost_ticket (core) 1—1 ost_autotask_ticket_map —┐< ost_autotask_sync_history
                                         └ (logical) Autotask Ticket
ost_autotask_sync_queue — outbound jobs keyed by osticket_ticket_id
ost_autotask_log         — cross-cutting audit/diagnostics
ost_autotask_settings    — runtime cursors & cached state
```